```
                context.Succeed(req);
            }
            return Task.CompletedTask;
        }
    }
}
```

Both handlers need to be registered with the service container:

```
builder.Services.AddSingleton<IAuthorizationHandler, IsInRoleHandler>();
builder.Services.AddSingleton<IAuthorizationHandler, HasClaimHandler>();
```

These registrations should replace the existing handler registration; otherwise, it will be evaluated as well. Handlers that implement `IAuthorizationHandler` are processed for every requirement, whereas those typed to a specific requirement only execute for that type of requirement.

To test this new arrangement out, remove the `Permission` claim from Anna, so she only has the `Admin` role and the `Joining date` claim. You can do this directly using a database management tool. Then run the application, and log in as her. You should find she is still able to access the Roles folder contents.

## *10.3.5 Roles or claims?*

We've explored two ways to establish whether a user is authorized: based on the role they are in or the claims they possess. But which approach should you favor? Roles have been part of ASP.NET since before the release of Identity, when the user management system revolved around a `Membership` framework. They are a pretty simple concept to grasp, but they are designed to work with groups of users and are intended to represent what those groups of users can do. The emphasis shifted to one that is claims-based when Identity was introduced. Claims describe what a user is,